

SA2DGE

SHREYAS'S ADVANCE 2 DIMENSIONAL GAME ENGINE

PROJECT STAGE 1 DOCUMENTATION

Game Development Fundamentals, Engine Architecture and the First Verified Direct3D 11 Graphics Runtime

Project Name	SA2DGE
Full Form	Shreyas's Advance 2 Dimensional Game Engine
Document Title	Project Stage 1 Documentation
Document Type	Technical project and architecture documentation
Author / Developer	Shreyas Pathak
Version	1.0 (Stage 1 Release)
Publication Date	September 2026
Technology Stack	C#, .NET, Windows Native Platform, Vortice, Direct3D 11, DXGI
Current Status	Phase 2 - Runtime Implementation (2.1 Runtime Bootstrap complete)

Abstract

SA2DGE is a proprietary two-dimensional game engine designed and developed from first principles in C# on the .NET platform, targeting Windows. Rather than relying on an existing engine, SA2DGE is being built so that the lower layer of game technology - the window, the game loop, timing, platform abstraction and the graphics pipeline - is understood and implemented directly.

This Stage 1 document is organised in two parts. **Part A** explains the core ideas of game development and game engine development. **Part B** documents SA2DGE itself: its vision, architecture, core systems, and the milestone that closes Stage 1 - a native Windows window whose back buffer is cleared to red through Direct3D 11 and presented through a flip-model DXGI swap chain, proving the complete path from C# code to the GPU and the screen.

PART A
Game Development and Engine Basics

PART B
The SA2DGE Project and Stage 1 Results

About This Document

Purpose

This document is the official Stage 1 record of the SA2DGE project. It captures what a game engine is, why it is built as a set of layered subsystems, how each of those ideas maps onto SA2DGE, and exactly what has been proven to work at the end of this stage. It is intended to serve three roles at once: a **learning record** of the concepts studied while building the engine, a **technical reference** for later stages, and a **formal project document** that can be reviewed, evaluated and extended.

Scope

Stage 1 covers the foundation of the engine: architecture, the Core runtime, the fixed-timestep game loop, the Windows platform window, native interop with Direct3D 11 through Vortice, and the first end-to-end rendering validation. Sprites, textures, cameras, ECS, physics, animation, audio and UI belong to later stages and appear here only in the roadmap.

Intended Audience

Developers who wish to understand how the engine is structured, evaluators assessing its technical depth, future contributors, and the author as a personal reference.

Document Structure

Part	Sections	Title and Summary
Part A	1 - 2	Game Development and the Game Engine. What a game is, the disciplines behind it, and what an engine provides.
Part A	3	The Game Loop and Time. Game loop, frame budget, delta time, fixed timestep and the accumulator.
Part A	4	Platform Layer and the OS Boundary. Platform abstraction, native windows, managed versus native code, native memory.
Part A	5 - 6	Graphics Fundamentals and Architecture Principles. CPU/GPU roles, Direct3D 11, DXGI, swap chains, render targets, layering.
Part B	7	SA2DGE Project Overview. Identity, vision, objectives, design principles and technology stack.
Part B	8	SA2DGE Architecture. Long-term subsystem stack, Core system, platform layer and graphics backend.
Part B	9	Stage 1 Implementation and Verification. Runtime flow, verified rendering pipeline, milestone checklist.
Part B	10 - 11	Current Phase, Roadmap and Conclusion. Phase 2 work packages, next steps, risks and closing summary.

Glossary of Abbreviations

Term	Meaning	Term	Meaning
API	Application Programming Interface	RTV	Render Target View
CPU	Central Processing Unit	dt	Delta time - time between two frames
GPU	Graphics Processing Unit	Hz	Hertz - cycles (updates) per second
D3D11	Direct3D 11, Microsoft graphics API	FPS	Frames per second
DXGI	DirectX Graphics Infrastructure	ECS	Entity Component System
.NET	Managed runtime used to run C# code	Interop	Communication between managed and native code
Vortice	.NET bindings for native graphics APIs	Backend	Implementation of an interface for one OS or API

PART A

Game Development and Game Engine Foundations

Before the SA2DGE project itself is described, this part builds the conceptual base it stands on: what games are made of, what a game engine provides, how time and the game loop work, how an engine talks to the operating system, and how pixels finally reach the screen.

1. Understanding Game Development

A video game is, at its core, a **real-time interactive program**. Unlike a document editor, which waits patiently for the user, a game keeps running whether or not the player does anything: it reads input, advances a simulated world, and draws a new picture of it many times every second. Everything a player experiences is the visible result of this cycle of input, simulation and presentation.

A game is normally described by the content and rules it presents to the player. These elements are the responsibility of the **game itself**, not of the technology beneath it:

- **Player** - the entity controlled by the human, with its abilities, state and controls.
- **Enemies** - autonomous entities that provide opposition, behaviour and challenge.
- **Levels** - the spaces, maps and scenarios in which play takes place.
- **Rules** - the conditions that define winning, losing, scoring, damage and progression.
- **Gameplay** - the moment-to-moment experience produced when all of the above interact.

1.1 The Disciplines Behind a Game

Discipline	Responsibility	Typical Output
Game Design	Defines mechanics, rules, difficulty, progression and the intended player experience	Design documents, level layouts, balance tables
Gameplay Programming	Implements the behaviour of players, enemies, items and rules on top of the engine	Player controller, enemy behaviour, scoring logic
Engine Programming	Builds the reusable runtime: loop, platform, rendering, resources, physics	Engine libraries and subsystems
Graphics Programming	Communicates with the GPU to draw sprites, text, effects and scenes	Renderer, render pipeline, shaders
Art, Animation and Audio	Creates visual assets, their motion, and music and sound effects	Sprites, tile sets, animation clips, sound banks
UI / UX and QA	Designs menus and interaction flow; tests stability and correctness	Menus, health bars, test plans, bug reports

2. What Is a Game Engine?

A **game engine** is a collection of reusable systems that provide the infrastructure required to build and run games. Instead of every game re-implementing windows, input, timing, rendering, audio and memory management, these responsibilities are written once and shared by every game built on top. The engine supplies the technology; the game supplies the content and decisions.

2.1 Engine Systems Versus Game Decisions

Engine System	What the Engine Provides	What the Game Decides
Window	An operating-system window as the drawing surface	Title, size and windowed or fullscreen mode
Input	Keyboard, mouse and controller events	What each key or button does in the game
Game Loop	The continuous cycle and its timing	What happens on every update

Engine System	What the Engine Provides	What the Game Decides
Rendering	Drawing images to the screen through the GPU	What is drawn, where and when
Audio	Sound loading, playback and mixing	Which sounds and music play, and when
Physics	Movement, forces and collision handling	Which objects collide and how they respond
Resources and Scenes	Loading and releasing assets; containers that organise game objects	Which assets each level needs and what it contains
Memory	Allocation and lifetime management	How many objects exist and when they appear or vanish

2.2 The Layered Model

A game never talks to the hardware directly. It calls the engine; the engine calls the operating system; the operating system controls the hardware. Every layer hides the complexity of the layer beneath it: the game supplies player, enemies, levels and rules; the engine supplies window, input, loop, rendering, audio, physics, resources, scenes and memory.



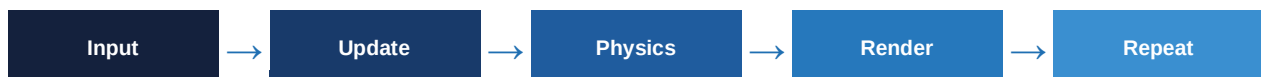
2.3 Why Build an Engine from First Principles?

Commercial engines are powerful, but they hide the lower layer. Building from first principles exposes it: how a window is created, a frame timed, a graphics device initialised, and a finished image delivered to the monitor. This changes how a developer reasons about performance and architecture, and lets the engine be shaped around its purpose - for SA2DGE, 2D games.

3. The Game Loop and Time

3.1 The Game Loop

The **game loop** is one of the most important concepts in game development. A game continuously performs the same cycle: read input, update the world, simulate physics, render the result, and repeat. Without a game loop the program would execute once and stop; with it, the game stays alive and responsive.



```

while (running)
{
    ProcessInput();
    Update();
    Render();
}
  
```

3.2 Frame Time and the Frame Budget

Every iteration of the loop is a **frame**, and each frame must finish inside a time budget. At 60 frames per second, input, logic, physics and rendering must fit inside roughly 16.67 milliseconds, or the player sees stutter. For reference: 30 FPS allows 33.33 ms per frame, 60 FPS allows 16.67 ms, and 120 FPS allows only 8.33 ms.

3.3 Delta Time

Games cannot assume that every computer runs at exactly the same speed. The engine therefore measures the time between two frames, known as **delta time (dt)**, and scales movement by it:

```
position += velocity x deltaTime
```

If a frame took 10 ms, deltaTime = 0.010 seconds, and a character moving at 200 units per second advances $200 \times 0.010 = 2$ units. On a slower machine with 20 ms frames it advances 4 units per frame but receives half as many frames, so the distance per second stays the same. Measuring frame time is a responsibility of the Core system, through its Time class, which every later subsystem can rely on.

3.4 Fixed Timestep and the Accumulator

Variable delta time is convenient but not always safe: a sudden long frame produces a large step, and large steps make physics unstable and behaviour unpredictable. A **fixed timestep** solves this by running the simulation at a constant rate regardless of how fast frames are rendered. SA2DGE uses a fixed rate of **60 Hz**, meaning 60 updates per second, so each update advances the simulation by $1 / 60 = 0.01667$ seconds (about 16.67 ms). Real elapsed time is collected in an **accumulator**, and the simulation consumes it in equal fixed slices:

```
accumulator += frameTime;
while (accumulator >= fixedStep)    // fixedStep = 1 / 60 s
{
    FixedUpdate(fixedStep);        // physics, deterministic logic
    accumulator -= fixedStep;
}
Render();
```

This pattern is particularly valuable for **physics**, **deterministic simulation** and **stable gameplay logic**, because the simulation behaves identically whether the machine renders 30 or 300 frames per second.

4. The Platform Layer and the Operating System Boundary

4.1 Why an Engine Needs a Platform Abstraction

An engine must create windows, receive input and talk to the operating system, but operating-system-specific code should not be scattered everywhere. If every subsystem called the Windows API directly, Windows details would spread through the whole code base. Instead, the engine talks to a **Platform Interface**, and a **backend** implements it for one operating system:



This keeps the architecture clean: the rest of the engine depends only on the interface, and all Windows-specific code lives in one place. This gives **isolation**, **maintainability** (a change to window creation touches one backend) and **flexibility** (another OS backend can be added behind the same interface).

4.2 The Native Window

A game needs an operating-system window, because the window provides the surface on which graphics eventually appear. It is also the gateway for essential events, so a complete window system handles **size and position**, **resize and close events**, **input events**, and **fullscreen and focus** changes. SA2DGE already creates its native window through the Windows backend.

4.3 Native Versus Managed Code

C# normally runs inside the managed .NET environment, but Windows and graphics APIs are native systems. Engine code therefore crosses a boundary, using **native interop** where necessary to reach Windows and Direct3D: C# calls into .NET, which calls the native API, which finally talks to Windows or the GPU. The two worlds follow different rules for memory and lifetime.

Aspect	Managed Memory (.NET)	Native Memory
Lifetime control	Handled automatically by the .NET Garbage Collector	Explicit and controlled through native lifetime management
How it is accessed	Through C# objects and references	Through pointers and handles
Typical use	Game logic, engine classes, data structures	Windows resources, graphics-API objects, GPU-related structures

4.4 Native Memory and Resource Lifetime

Graphics APIs frequently require structures and resources that interact closely with native memory. Engine development therefore requires a clear understanding of **pointers**, **handles**, **memory ownership**, **resource lifetime and disposal**. Incorrect lifetime management causes memory leaks, invalid handles, access violations and GPU resource errors, so every native resource must have a clear owner and release point.

5. Graphics Programming Fundamentals

5.1 The Roles of the CPU and the GPU

Rendering is a collaboration between two processors. The **CPU** runs game logic, input, physics and scene updates and prepares rendering commands; the **GPU** performs vertex processing, rasterization and other massively parallel graphics work, rendering into a framebuffer shown on screen.



5.2 Direct3D 11, DXGI and the Presentation Chain

Direct3D 11 (D3D11) is Microsoft's graphics API for communicating with GPUs on Windows. Through it an application tells the GPU to create resources, create the rendering pipeline, upload data, draw and present frames. **DXGI** supplies the infrastructure around graphics devices and presentation, most importantly the swap chain. Key concepts:

Concept	Role in Rendering	Simple Analogy
Graphics API (D3D11)	Software interface used to instruct the GPU	The language spoken to the GPU
Device	Creates and manages GPU resources: buffers, textures, shaders and views	The workshop that makes the tools
Device Context	Issues rendering commands that use those resources	The worker operating the tools
DXGI	Infrastructure for graphics devices and presentation	The delivery route to the display
Swap Chain	Manages the buffers that are presented to the screen	A set of canvases taking turns on display
Back Buffer	The hidden image that is currently being drawn	The canvas being painted out of sight
Render Target View (RTV)	Lets the pipeline use the back buffer texture as a render target	A "paint here" label on the canvas
Clear	Fills the entire render target with one colour	Painting the whole canvas one colour
Present	Submits the completed frame for display	Revealing the finished canvas

5.3 The Flip-Model Swap Chain

Instead of drawing onto the physical monitor, a game renders each frame into a **back buffer** and then **presents** it. SA2DGE uses a **flip-model swap chain**, the modern Windows presentation approach designed for efficient presentation of rendered frames. Because the back buffer is only a texture, the pipeline writes to it through a **Render Target View**: Swap Chain, Back Buffer, Render Target View, Output Merger, Pixels.

6. Engine Architecture Principles

Five principles keep a growing engine from collapsing into a tangle and guide every SA2DGE decision:

Principle	Meaning	Benefit
Separation of Concerns	Each subsystem has one clear responsibility	Easier to understand, test and replace
Layered Dependencies	Upper layers use lower layers, never the reverse	Predictable, acyclic structure
Abstraction Boundaries	Subsystems meet through interfaces	Backends change without engine rewrites
Explicit Ownership	Every resource has an owner	Fewer leaks and invalid handles
Incremental Verification	Each layer is proven before the next	Bugs are found close to their cause

Together, these principles explain the shape of the SA2DGE architecture described in Part B.

PART B

The SA2DGE Project

Part B applies the concepts of Part A to the actual engine. It presents the identity and vision of SA2DGE, its long-term architecture, the Core runtime, and the verified rendering pipeline that completes Stage 1.

7. SA2DGE Project Overview

7.1 Project Identity

Name	SA2DGE
Expansion	Shreyas's Advance 2 Dimensional Game Engine
Author / Developer	Shreyas Pathak
Technology	C# on .NET; Windows native backend; Direct3D 11 and DXGI through Vortice
Development Method	Built subsystem by subsystem; each layer verified before the next begins
Current Status	Phase 2 - Runtime Implementation; Stage 1 milestone reached

7.2 Vision and Objectives

The vision of SA2DGE is a complete, understandable and extensible proprietary 2D game engine in which every layer - from the operating-system window to the final sprite - is written deliberately. The objectives of the project are:

- **Understand the lower layer** - implement the window, loop, timing and graphics foundations directly instead of importing them.
- **Build a clean architecture** - separate subsystems with clear responsibilities and minimal coupling.
- **Reach a real 2D renderer** - progress from a cleared frame to sprites, textures and cameras, and finally a first real 2D scene.
- **Create a foundation for games** - provide the systems on which complete 2D games can later be developed.

7.3 Guiding Design Principles

SA2DGE applies the principles of Section 6 as concrete rules. **Each subsystem should have a clear responsibility and should not unnecessarily depend on the internal implementation of another subsystem** - this is separation of concerns. Platform and Graphics are hidden behind abstractions so the renderer does not depend directly on Direct3D-specific code, native resources have explicit owners, and development is incremental: nothing is finished until it works on screen.

7.4 Technology Stack

Layer	Technology	Role in SA2DGE
Language	C#	Primary language of the engine and its subsystems
Runtime	.NET	Managed execution environment for engine code
Platform	Windows native backend	Creates the native window and connects the engine to the operating system
Native Bindings	Vortice	.NET bindings that expose native graphics functionality to C#
Graphics API	Direct3D 11	Rendering backend used to communicate with the GPU
Presentation	DXGI, flip-model swap chain	Swap chain and presentation of finished frames to the display
Timing	Fixed 60 Hz timestep	Predictable update rate: 1 / 60 s, about 16.67 ms per update

8. SA2DGE Architecture

8.1 The Long-Term Subsystem Stack

The engine is developed as separate subsystems. The table lists the long-term architecture with each subsystem's intended responsibility and its status at the end of Stage 1:

#	Subsystem	Intended Responsibility	Stage 1 Status
1	Core	Engine lifetime: start-up, loop, time, configuration, errors, shutdown	Bootstrap complete
2	Platform	Separates the engine from the OS: windows, events, native handles	Window proven
3	Graphics	Rendering backend on Direct3D 11 and presentation to the screen	Pipeline proven
4	Math	Vectors, matrices and transforms for 2D space	Planned
5	Scene	Organises everything that exists in the game world	Planned
6	ECS	Entity-component data model for game objects	Planned
7	Resources	Loading, ownership and lifetime of assets	Planned
8	Physics	Movement, collision detection and response	Planned
9	Animation	Time-based change of sprites and properties	Planned
10	Audio	Sound and music playback	Planned
11	UI	In-game interface elements	Planned
12	Serialization / Tools	Saving and loading data; supporting tools and utilities	Planned

8.2 The Core System

The **Core** is responsible for controlling the lifetime of the engine. Conceptually, the Application owns the Engine, the Engine drives the GameLoop, and the GameLoop runs the Game:



Core Class	Responsibility
Application	Represents the actual program running on the operating system; starts and coordinates the engine rather than implementing every subsystem itself
Engine	Represents the engine itself; coordinates the major runtime systems and controls their lifetime
Game	Represents the game running inside the engine; uses engine technology and decides what is rendered, where and when
GameLoop	Drives the repeated cycle of input, update and render while the engine is running
Time	Measures delta time and supports the fixed 60 Hz timestep
EngineConfig	Holds the configuration values with which the engine is started
EngineException	Engine-specific error type used to report failures clearly

8.3 Application Lifecycle

The Application's responsibility is intentionally small - it starts the engine and the game, runs the loop and shuts down in an orderly way:



8.4 The Platform Layer and the Graphics Backend

Two abstractions carry Stage 1. The **Platform layer** hides the operating system: SA2DGE talks to a Windows Platform, which creates a native window and communicates with Windows. The **Graphics Backend** hides the graphics API: the Renderer talks to the backend, which uses Direct3D 11 to reach the GPU, so renderer components need not depend directly on Direct3D-specific code.



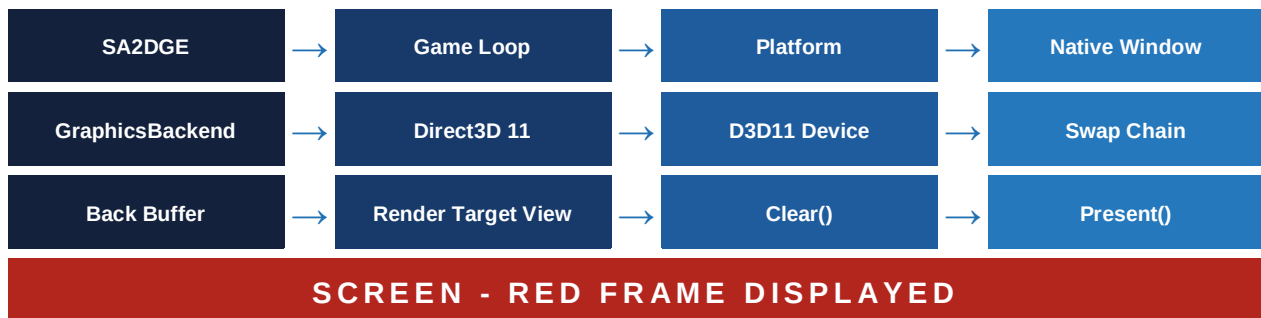
9. Stage 1 Implementation and Verification

9.1 Implementation Summary

Stage 1 followed the rule of incremental verification: each step was proven before the next was added. The .NET runtime and Core were brought up first, then the fixed 60 Hz game loop, then the native Windows window through the platform backend. Only then did Vortice create the D3D11 device, a flip-model DXGI swap chain was attached to the window, and a Render Target View was created from the back buffer. Finally Clear() filled the target with red, Present() displayed the frame, and the runtime shut down cleanly.

9.2 The Verified Rendering Pipeline

This is the most important diagram of Stage 1. Every box in it is working in SA2DGE:



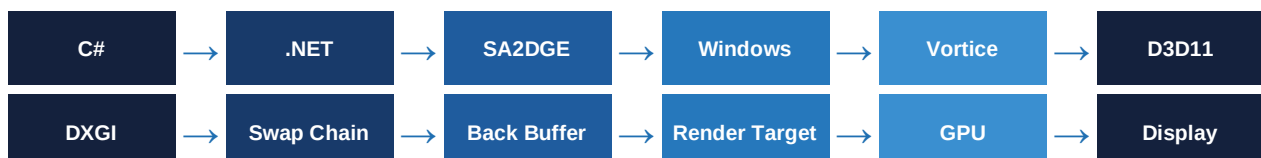
9.3 Milestone Checklist

The following capabilities have been demonstrated end to end at the close of Stage 1:

Capability	Status	Capability	Status
.NET runtime	Complete	Back buffer	Complete
Engine initialisation	Complete	Render target	Complete
Game loop	Complete	Clear operation	Complete
Fixed 60 Hz timestep	Complete	Present operation	Complete
Windows native window	Complete	Actual red frame on screen	Complete
Native platform backend	Complete	Clean runtime shutdown	Complete
Vortice integration	Complete	D3D11 device creation	Complete
DXGI integration	Complete	Flip-model swap chain	Complete

9.4 Why the Red Frame Matters

A solid red window looks simple, but it proves a great deal. For it to appear, the whole chain must work: C#, .NET, SA2DGE, Windows, Vortice, D3D11, DXGI, swap chain, back buffer, render target, GPU and display. It is the **first major end-to-end graphics validation**, and it marks the point at which SA2DGE crossed from an architectural project into an actual working graphics runtime.



10. Current Phase and Roadmap

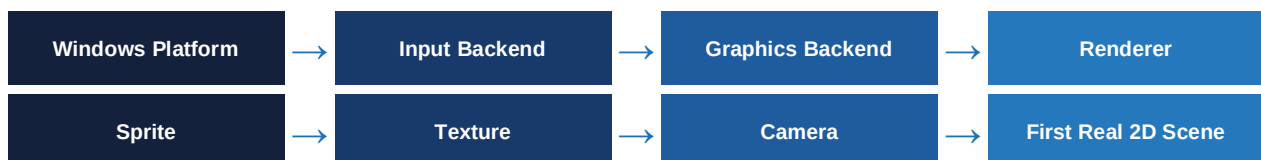
10.1 Phase 2 - Runtime Implementation

ID	Work Package	Status	Focus
2.1	Runtime Bootstrap	Essentially complete	Engine start-up, loop, timing and first graphics validation
2.2	Windows Platform	Next	Make the platform layer properly usable by the engine
2.3	Input Backend	Planned	Deliver input from the platform to the engine
2.4	Graphics Backend	Planned	Formal backend abstraction over D3D11
2.5	First Real 2D Renderer	Planned	Sprites, textures and camera
2.6	Runtime Integration	Planned	Connect all subsystems into one runtime
2.7	Tests and Validation	Planned	Verify each layer and the whole runtime

10.2 The Immediate Next Step

The next step is **not** to rebuild D3D11 - Window, D3D11, Swap Chain, Render Target, Clear and Present already form a working graphics runtime. Work now moves to making the Windows Platform layer properly usable: window creation, destruction, resize, close, events, focus, position, size and native handle.

10.3 The Path to the First Real 2D Scene



Each step is verified on screen before the next begins, as in Stage 1.

10.4 Risks and Their Management

Risk	Why It Matters	Approach
Native lifetime errors	Leaks and invalid handles can crash the runtime or corrupt GPU work	Explicit ownership and disposal for every native resource
Platform coupling	Windows-specific code spreading through the engine	Keep all OS code behind the Platform interface

11. Conclusion

Stage 1 has established the foundation of a layered, professionally structured 2D game engine. The concepts of Part A - game loop, fixed timestep, platform abstraction, native interop and the Direct3D 11 pipeline - are now working code, ending in a real window, a real GPU pipeline and a real frame on screen. Development now continues with Work Package 2.2, the Windows Platform layer.

The One Big Concept to Remember

A game engine is a layered system. The game does not directly talk to the GPU: it goes through the engine, platform abstraction, graphics backend, graphics API, and finally the GPU. **GAME, ENGINE, PLATFORM, GRAPHICS BACKEND, DIRECT3D 11, GPU, SCREEN** - this whole path has been completed for the first basic frame, and it is the foundation on which the actual 2D renderer will be built.

Written by

SHREYAS PATHAK

Author / Developer of SA2DGE - Shreyas's Advance 2 Dimensional Game Engine

